

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут інформаційно-комунікаційних технологій та електронної інженерії
Кафедра «Інформаційно-комунікаційних технологій»



Звіт з лабораторної роботи №8
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи IX-21
Петриченко Сергій

Прийняла:
док. філософії, доцент
Гордійчук-Бублівська О.В.

Львів 2026 р.

Тема роботи: Реалізація клієнт-серверної взаємодії з використанням протоколу MQTT, автоматизація передачі даних через API.

Мета роботи: Розробити клієнт-серверну архітектуру з використанням MQTT протоколу, реалізовувати класи для автоматизації роботи з API.

Теоретичні відомості

Основи протоколу MQTT.

MQTT (Message Queuing Telemetry Transport) – це легкий протокол обміну повідомленнями, спеціально розроблений для комунікації в умовах обмеженої пропускної здатності мережі та для пристроїв із низьким рівнем обчислювальних ресурсів. Він є стандартом де-факто для Інтернету речей (IoT). Архітектура MQTT базується на шаблоні "публікація-підписка" (publish-subscribe), що виключає прямий зв'язок між відправником та одержувачем.

Ключовим елементом системи є MQTT-брокер – центральний сервер, який приймає всі повідомлення від клієнтів (видавців) і маршрутизує їх іншим клієнтам (підписникам) на основі ієрархічної системи "тем" (topics). Це забезпечує високу масштабованість та асинхронність обміну даними.

Інтеграція сучасних вебтехнологій: REST API та WebSocket.

Для побудови комплексних систем передачі даних MQTT часто комбінують з іншими протоколами:

REST API: Використовує класичні HTTP-запити (GET, POST тощо) для одноразового отримання або відправки структурованих даних (найчастіше у форматі JSON). Це зручний інструмент для взаємодії з базами даних або отримання статичної конфігурації.

WebSocket: Забезпечує постійне повнодуплексне з'єднання між клієнтом і сервером поверх одного TCP-з'єднання. На відміну від REST, де ініціатором завжди є клієнт, WebSocket дозволяє серверу миттєво відправляти дані клієнту, що критично важливо для роботи в режимі реального часу.

Синергія технологій.

У сучасних архітектурах REST API використовується для первинного збору або конфігурації параметрів, WebSocket – для швидкої двосторонньої трансляції масивів інформації, а MQTT – для надійної доставки результатів або керівних команд до кінцевих пристроїв та датчиків із дотриманням заданих рівнів якості обслуговування (QoS).

Результати виконаної роботи

```
File Edit Selection View Go Run Terminal Help OOP-Labs
main.py X
main.py > ...
1. Розробити MQTT клієнт MQTTClient, який дозволяє публікувати повідомлення на задану тему на реальному MQTT брокері.
Вимоги до завдання:
• Створити клас MQTTClient, який містить методи:
  o connect(): для підключення до MQTT брокера.
  o publish(topic, message): для публікації повідомлення на задану тему.
  o disconnect(): для відключення від брокера.
• Використовувати реальний MQTT брокер для тестування:
  o Наприклад, можна використовувати сервіси:
    1. CloudMQTT – безкоштовний MQTT брокер з обмеженням на кількість підключень.
    2. HiveMQ – безкоштовний сервіс для тестування MQTT.
  o Зареєструйтесь на одному з цих сервісів та отримайте необхідні дані для підключення до брокера (адреса сервера, порт, логін, пароль).
  o Можна використати сервіси без реєстрації, наприклад broker.hivemq.com
• Інструкція по підключенню до MQTT брокера:
  o Зареєструйтесь на обраному MQTT сервісі (наприклад, CloudMQTT або HiveMQ).
  o Отримайте наступні дані:
    1. Адреса сервера (наприклад, mqtt://mqtt.eclipse.org).
    2. Порт для підключення (наприклад, 1883 для незахищеного з'єднання або 8883 для захищеного через TLS/SSL).
    3. Тема для публікації (наприклад, home/temperature).
  o Використовуйте ці дані для налаштування підключення в класі MQTTClient.
2. Інтеграція всіх компонентів в єдину систему:
• Створіть програму, яка:
  o Отримує дані через REST API.
  o Передає отримані дані через WebSocket.
  o Публікує ці дані через MQTT.
• Вибрані API сервіси повинні працювати разом для передачі даних в режимі реального часу.

import paho.mqtt.client as mqtt
import time
import requests
import websocket
import json
import ssl

class MQTTClient:
    def __init__(self, broker_address, broker_port, username, password):
        self.broker_address = broker_address
```

```
File Edit Selection View Go Run Terminal Help OOP-Labs
main.py X
main.py > ...
class MQTTClient:
    def __init__(self, broker_address, broker_port, username, password):
        self.broker_port = broker_port
        self.client = mqtt.Client()
        self.client.username_pw_set(username, password)
        self.client.tls_set(tls_version=ssl.PROTOCOL_TLS)

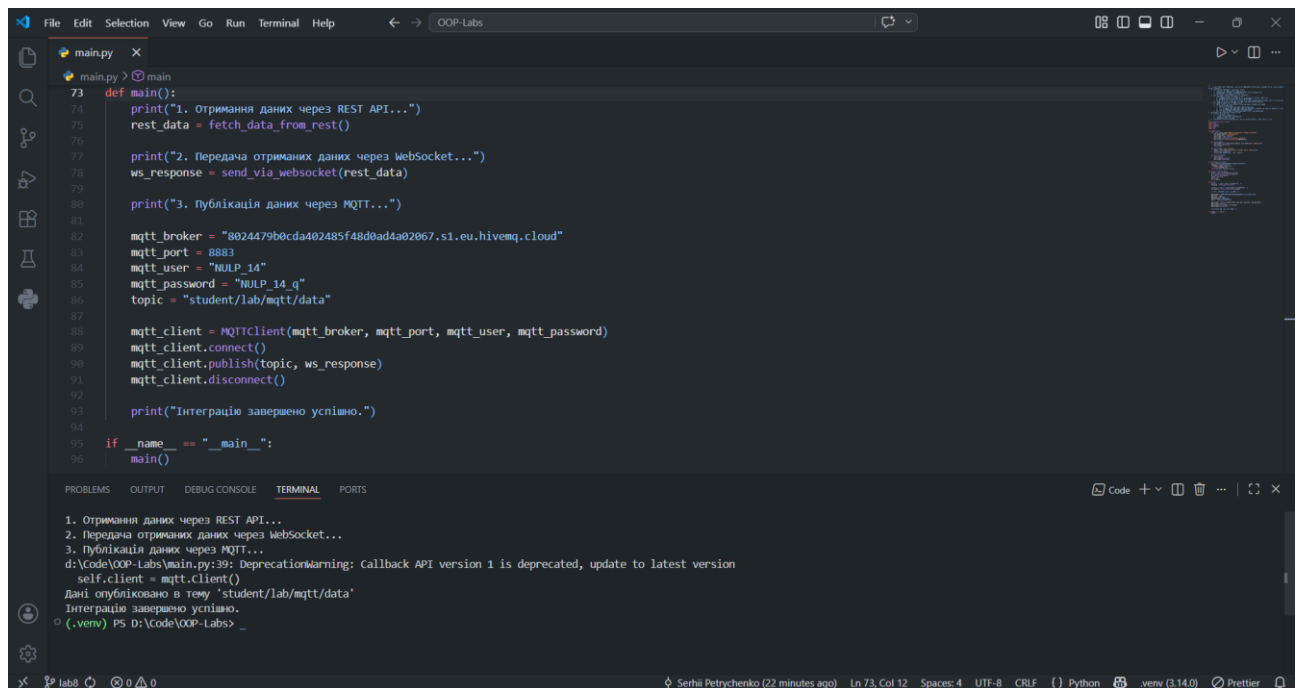
    def connect(self):
        self.client.connect(self.broker_address, self.broker_port, keepalive=60)
        self.client.loop_start()
        time.sleep(1)

    def publish(self, topic, message):
        result = self.client.publish(topic, message, qos=0, retain=False)
        result.wait_for_publish()
        print(f"Дані опубліковано в тему '{topic}'")

    def disconnect(self):
        time.sleep(1)
        self.client.disconnect()
        self.client.loop_stop()

    def fetch_data_from_rest():
        url = "https://jsonplaceholder.typicode.com/posts/1"
        response = requests.get(url)
        if response.status_code == 200:
            return response.json()
        return {"error": "Failed to fetch"}

    def send_via_websocket(data):
        ws_url = "ws://ws.postman-echo.com/raw"
        ws = websocket.create_connection(ws_url)
        ws.send(json.dumps(data))
        result = ws.recv()
        ws.close()
        return result
```



The screenshot shows a VS Code editor window with a Python file named `main.py`. The script defines a `main()` function that performs three steps: 1. Fetching data from a REST API, 2. Sending the data via WebSocket, and 3. Publishing the data via MQTT. The MQTT configuration includes a broker address, port 8883, user 'NULP_14', password 'NULP_14_q', and topic 'student/lab/mqtt/data'. The terminal output shows the successful execution of these steps, with a deprecation warning for the MQTT client version.

```
def main():
    print("1. Отримання даних через REST API...")
    rest_data = fetch_data_from_rest()

    print("2. Передача отриманих даних через WebSocket...")
    ws_response = send_via_websocket(rest_data)

    print("3. Публікація даних через MQTT...")

    mqtt_broker = "8024479b0cda402485f48d8ad4a02067.s1.eu.hivemq.cloud"
    mqtt_port = 8883
    mqtt_user = "NULP_14"
    mqtt_password = "NULP_14_q"
    topic = "student/lab/mqtt/data"

    mqtt_client = MQTTClient(mqtt_broker, mqtt_port, mqtt_user, mqtt_password)
    mqtt_client.connect()
    mqtt_client.publish(topic, ws_response)
    mqtt_client.disconnect()

    print("Інтеграцію завершено успішно.")

if __name__ == "__main__":
    main()
```

Terminal Output:

```
1. Отримання даних через REST API...
2. Передача отриманих даних через WebSocket...
3. Публікація даних через MQTT...
d:\Code\OOP-Labs\main.py:39: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  self.client = mqtt.Client()
Дані опубліковано в тему 'student/lab/mqtt/data'
Інтеграцію завершено успішно.
(.venv) PS D:\Code\OOP-Labs>
```

Висновки

1. Розроблено програмний сценарій мовою Python для автоматизації передачі даних між різними мережевими інтерфейсами. Було реалізовано клієнт-серверну взаємодію, у якій інформація послідовно завантажується через REST API, транслюється через постійне з'єднання WebSocket, а потім публікується на захищеному хмарному брокері HiveMQ з використанням протоколу MQTT та TLS-шифрування.
2. Така архітектура є оптимальною для побудови розподілених систем збору та обробки даних. Наприклад, її можна ефективно застосувати для віддаленого опитування засобів вимірювальної техніки. Дані з датчиків, що містять експериментальні значення із заданою похибкою 2-3%, можуть передаватися до центрального сервера для розрахунку метрологічних характеристик, перевірки класів точності або побудови спектральних діаграм смугових сигналів у режимі реального часу.
3. Виконання роботи дозволило закріпити навички програмування мережевої взаємодії та глибше зрозуміти механізми асинхронної передачі інформації. Було отримано практичний досвід роботи з багатоетапними пайплайнами, що дає розуміння того, як надійно збирати цифрові сигнали та маршрутизувати їх перед застосуванням алгоритмів математичного чи спектрального аналізу.
4. Переваги: Висока гнучкість системи, здатність працювати в режимі реального часу (завдяки WebSocket), низьке навантаження на канал зв'язку при передачі телеметрії (завдяки MQTT), а також надійна ізоляція компонентів системи один від одного.
Недоліки: Ускладнення загальної кодової бази через необхідність підтримувати три різні протоколи. Крім того, наявність зовнішнього хмарного брокера (HiveMQ) створює залежність від стабільності інтернет-з'єднання та вимагає додаткових налаштувань безпеки для захисту каналів передачі.